Cross-Environment TPC-H Runtime Prediction (CETP)

CETP predicts how long a TPC-H analytical query will take on a production machine, using only a query plan captured on a development machine and static hardware specifications for both machines. The problem it addresses is that query performance does not transfer across hardware in a simple way: a query that is fast in development can be slow in production, or vice versa, and running a full benchmark suite on every candidate production machine before every deployment is expensive. CETP replaces that benchmark with a prediction, so a CI/CD pipeline can flag a likely SLA violation before a query ever reaches production.

How it works

CETP decomposes the prediction into two independent parts. The first is the workload cost, extracted from a PostgreSQL `EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON)` plan on the development machine: planner cost, shared buffer hits and reads, and row counts. The second is the hardware signature, a pair of static scores per machine, memory bandwidth and multi-core compute throughput, measured once via short microbenchmarks. A Random Forest classifier uses the workload cost to estimate whether a query is bandwidth-bound, compute-bound, I/O-bound, or mixed, and a set of gradient-boosted quantile regressors combine that classification with the ratio of the two machines' hardware signatures to predict the production runtime's p50, p95, and p99. A hardware-distance check flags predictions made for a production machine that lies far outside the training set, so the system can decline to guess rather than answer confidently on unfamiliar hardware.

Headline result

All numbers about this project come from `results/CANONICAL.md`, regenerated by `make results`. Do not quote figures from any other file — several older reports in `results/` predate the buffer-bug correction or the multi-core compute fix and are marked superseded where so.

> **Active work (2026-10-13): the predictor described below is being replaced.** The one experiment that tested the project's actual claim — predict production runtime from a developer's machine, arm64 laptop → x86 EC2 — scored **324% MAPE against 123% for assuming production equals development.** It was worse than doing nothing. The replacement measures capability by running PostgreSQL's own operators in SQL (so it also works on RDS/Aurora, which the C probe could not), scales per operator, and takes the plan from production. Design, status and what is still unvalidated: **OPERATOR_MODEL.md**.

Across 5 EC2 instance types and 21 TPC-H queries, a single bandwidth ratio is a strong baseline that no more complex approach beat by a statistically significant margin, with one exception: once the compute microbenchmark was corrected to measure multi-core throughput instead of single-core throughput, the physics-based formula using it significantly outperformed the baseline — **21.61% MAPE vs**

23.05%, improvement +1.44pp, 95% cluster-bootstrap CI **[0.32, 2.56]**, excluding zero. That figure is computed on the buffer-corrected dataset with the bottleneck classifier refit inside every cross-validation fold.

Read that result with its qualification. Scaling by `1/compute_ratio` alone — no bottleneck classifier, no roofline weighting, no fitted parameters — scores **20.06% MAPE**, *better* than the analytical formula's 21.61%. The paired difference is +1.55pp with CI **[-0.41, 3.51]**, which includes zero, so neither is significantly better than the other. The honest reading is that **the classifier and the roofline weighting are statistically indistinguishable from their own simplest component**: the gain over the bandwidth baseline comes from using *compute* rather than *bandwidth* as the scaling signal, not from the model built on top of it. Any summary of this project that reports 21.61% without reporting 20.06% beside it is misleading.

The mechanism is confirmed rather than assumed: PostgreSQL chooses parallel plans, so the compute resource actually consumed is *aggregate* multi-core throughput, which the single-core probe could not see (r = 0.458, p = 0.037 — `results/fix3_parallelism_mechanism_summary.csv`).

A second finding emerged from the leakage-free re-evaluation: **the analytical formula's accuracy is protocol-invariant** (21.61% / 21.66% / 21.64% under leave-one-machine-out, leave-one-query-out, and both at once) **while the learned model's swings from 17.29% to 26.86% depending purely on whether it has seen the target hardware.** Given every machine, the learned model is the most accurate approach by a wide margin; given a new machine, it is worse than the baseline. It interpolates across hardware; it does not extrapolate.

Repository structure

```
cetp/
  README.md              this file
  ROADMAP.md             audit findings and the plan to turn this into research + product
  TASKS.md               the prioritised task list behind that plan
  DOC_CORRECTIONS.md     corrections required in the external deck and LLD
  Makefile               `make results` regenerates every published number
  QUICKSTART.md          how to use the tool
  IMPLEMENTATION_REFERENCE.md  what is implemented, vs what the LLD claims
  CONFERENCE_PLAN.md     the path to a publishable paper, at zero cost
  PREREGISTRATION.md     analysis plan committed before the next data collection
  DATASHEET.md           dataset documentation (Gebru et al. structure)
  TIER1_RESULTS.md / TIER2_RESULTS.md / TIER3_RESULTS.md   findings by phase
  cetp/                  the installable package: predict, regress, redact, signature, cli
  gate/                  cetp_gate.py (SLA gate CLI), cetp_gate_demo.py, cetp.yml, example
queries
  site/                   single-file project page (site/index.html), hostable as-is
  models/                 final trained model artifacts (classifier, quantile regressors,
hardware signatures)
  models_online/          versioned self-calibration snapshots from the online learning
addition
  data/
    raw_buggy_do_not_use/ quarantined original dataset; buffer counts inflated 3-12x by a
fixed bug
    corrected/            tpch_dataset_corrected_v2.csv is THE canonical dataset; every
script reads it
  scripts/
    collection/            TPC-H collection harness, patched queries, compute/bandwidth
microbenchmarks
    analysis/               phase and addition scripts that produced every number in results/
  tests/                  test_e2e.py and its EXPLAIN-plan fixtures
  results/                research reports, evaluation metrics, and figures, organized by
phase and addition
  .github/workflows/     CI gate that runs cetp_gate.py against example queries on every push
```

Using it

```
pip install -e .
cetp selftest
```

Then the recommended mode — compare a query before and after a change, which cancels most of the cross-machine model error:

```
cetp regress --before before.json --after after.json --dev c7i --prod c5a --fail-on-
regression
```

Full usage, including absolute prediction, plan redaction and registering a new machine, is in QUICKSTART.md. What is implemented versus what the design documents claim is in IMPLEMENTATION_REFERENCE.md.

Before shipping a plan anywhere, strip its literals — `EXPLAIN` output embeds the query's values, which on a real database is user data:

```
cetp redact plan.json -o plan.redacted.json
```

Regenerating the results

```
make install   # pinned dependencies; scikit-learn is pinned exactly because models/*.pkl are
version-coupled
make results   # -> results/CANONICAL.md + results/canonical_metrics.json
make verify-determinism   # asserts two runs are byte-identical
make test      # the gate test suite
```

Running the gate demo

From the repository root:

```
python gate/cetp_gate.py --demo
```

This runs four worked scenarios covering every verdict path (PASS, BLOCK, WARN, and WARN for low confidence) and writes a report to `results/phase8_cetp_gate_demo.md`. To run the gate against a single query directly:

```
python gate/cetp_gate.py --explain-json path/to/explain.json --config gate/cetp.yml --query-
id q1
```

To see the online self-calibration loop, which detects an unfamiliar machine, learns from 12 measurements, and cuts its error on that machine from 77.7% to 15.0%:

```
python scripts/analysis/addition7_online_loop.py
```

To read the project page, open `site/index.html` in a browser, or host that one file anywhere. It is self-contained: no external requests, no build step. Every figure on it comes from `results/`.

The old `dashboard/` was removed on 2026-10-13: it was a static snapshot of superseded numbers (21.60% headline, no `cmp_only`, none of the per-operator work) and `site/index.html` replaces it. Recover it from git history if needed.

```

Then open `http://localhost:8000`.

To run the test suite:
```

pytest tests/test_e2e.py

Further detail

The full writeup, including the complete experimental methodology, every addition and fix made after the initial results, and the paper itself, is maintained outside this repository. Within this repository, `results/phase7_report.md` is the most complete single account of the final method and its evaluation, and each other file in `results/` documents one phase or addition referenced from there.

Honest status

The research is sound; treat the product as alpha. What works: prediction, regression detection, redaction, machine probing, and a CI action, all exercised by `cetp selftest` and 28 tests. What does not: absolute SLA gating at the accuracy available (54.5% of absolute BLOCK verdicts in evaluation were false alarms), and any coverage guarantee on hardware outside the calibration set.

The `models/*.pkl` artifacts that `gate/cetp_gate.py` loads are **stale** — see models/MODEL_CARD.md. The installable `cetp` package does not use them: it ships the analytical formula plus a calibration table, which is both more accurate out of hull and immune to the staleness.

Limitations

- **5 machines, 21 queries, SF1, EBS-only, no concurrency.** Nothing here speaks to other hardware, instance-local NVMe, other scale factors, or concurrent workloads. The `io` bottleneck class reflects buffer-touch patterns under one uniform storage backend, not storage diversity.
- **The compute signal is only usable in multi-core form.** Single-core correlates at the physically wrong sign (+0.213 vs −0.691) and makes predictions worse.
- **Every p95/p99 result published before 2026-10-12 is contaminated.** Repetition 1 of each measurement cell is a cold-cache run — the slowest of 20 in 61 of 105 cells, by up to 8.33x — and at n=20 the 95th/99th percentiles sit exactly where that run lands. The p50 headline is unaffected (worst-case 0.7%), but tail targets, the quantile calibration table, and any p99-based gate result must be regenerated. `make results` now discards the warmup repetition.
- **The shipped gate does not embody the headline finding.** `models/hardware_signature.pkl` and `gate/cetp.yml` still carry the old single-core compute scores.
- **The bottleneck classifier is weak and its labels are opinions.** Macro-F1 0.375 on the corrected dataset with `io` recall 0.000, against a 0.161 trivial baseline. The 4 class labels are 21 hand-assigned judgement calls, and because all six classifier features are machine-invariant it is effectively a 21-row lookup table.
- **The three-state gate is not an automatic stop.** It blocks ~8% of pairs with a 54.5% false-alarm rate, and its PASS state covered 4 of 840 pairs.

`results/phase7_report.md` has the fullest discussion of the earlier negative results, but its headline conclusion is superseded — see the banner at the top of that file. Everything current is in `results/CANONICAL.md`.

Status

Academic capstone project.